

UNIVERSITY OF ASIA PACIFIC
DEPARTMENT OF CSE

Lab Report - 7

COURSE CODE: CSE 402.

COURSE TITLE: OPERATING SYSTEM LAB.

SUBMITTED BY:

NAME : SAZID ABDULLAH

STUDENT ID : 23101146

SEMESTER : 4th YEAR 1st SEMESTER

SECTION : C - 2

DATE : 10/12/2026

SUBMITTED TO

ATIA RAHMAN ORTHI

LECTURER AT UAP

1. Problem:

Given the following disk request queue and initial disk head position:

- Request Queue: [3, 60, 10, 25, 30, 4]
- Initial Head Position: 41

Implement the following Disk Scheduling Algorithms:

- Shortest Seek Time First (SSTF)
- First Come First Serve (FCFS)

For each algorithm, calculate the total disk head movement and represent the request servicing sequence. Finally, compare the performance of SSTF and FCFS based on total head movement and determine which algorithm is more efficient for the given request queue.

2. Source Code:

```
requests = [3, 60, 10, 25, 30, 4]
total_head_movement = 0
seek_seq = []
current_head = 41

pending_requests = list(requests)

while pending_requests:
    closest_req = min(pending_requests, key=lambda req:
abs(current_head - req))
    distance = abs(current_head - closest_req)
    total_head_movement += distance
    current_head = closest_req
    seek_seq.append(closest_req)
    pending_requests.remove(closest_req)
print("Total_head_movement:", total_head_movement)
print("Seek Sequence:", seek_seq)

print("FCFS")

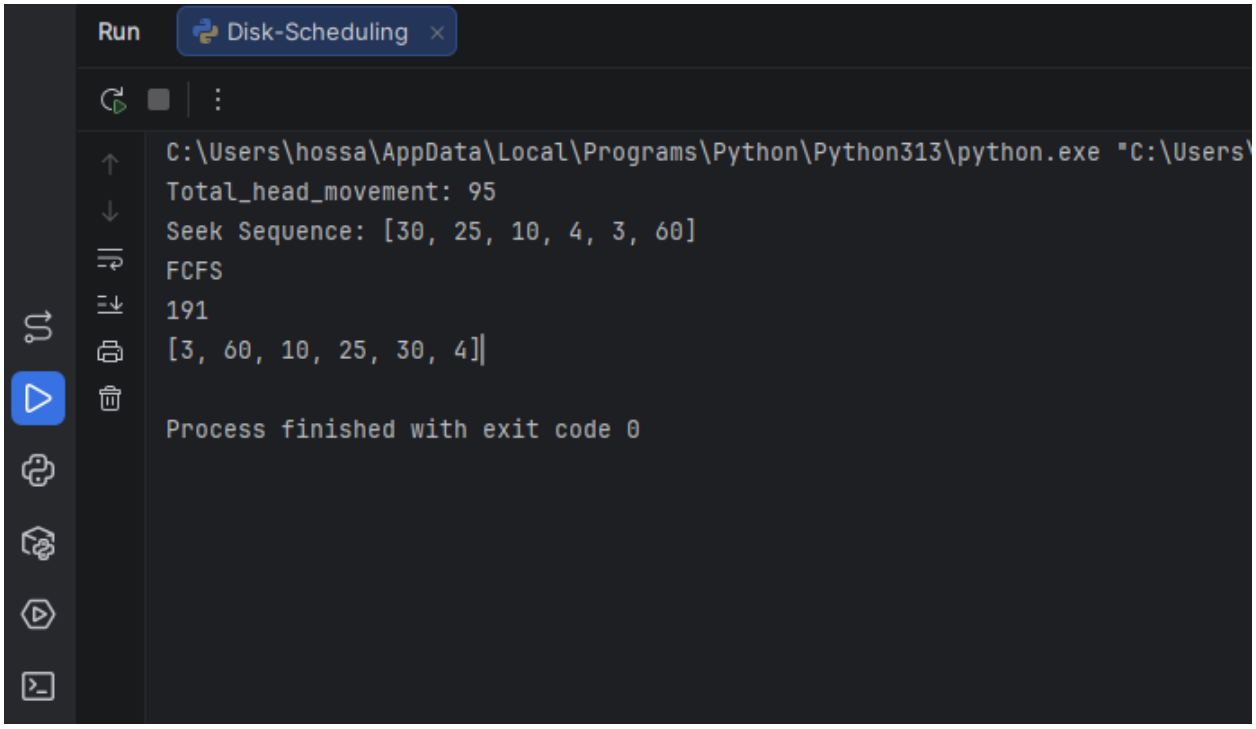
total_head_movement = 0
current_head = 41
seek_seq = []

for requests in requests:
```

```
    distance = abs(current_head - requests)
    total_head_movement += distance
    seek_seq.append(requests)
    current_head = requests

print(total_head_movement)
print(seek_seq)
```

3. Output:



4. Discussion:

In this experiment, the FCFS and SSTF disk scheduling algorithms were analyzed using an identical set of disk requests and the same initial head position. Their performance was compared by calculating the total distance traveled by the disk head.

The experimental results indicate that FCFS produces a total head movement of 191 tracks, while SSTF requires only 95 tracks. This difference occurs because FCFS handles requests according to their arrival order, regardless of their

distance from the current head position. On the other hand, SSTF prioritizes the nearest pending request, which helps minimize unnecessary head movement.

Compared with FCFS, SSTF saves 96 tracks of movement, achieving an approximate reduction of 50.26%. These findings demonstrate that SSTF is more efficient than FCFS for the given set of disk requests in terms of total head movement.

Conclusion:

The experiment demonstrated the working principles and performance differences between the FCFS and SSTF disk scheduling algorithms. Based on the calculated results, SSTF completed all requests with 95 tracks of head movement, whereas FCFS required 191 tracks.

Thus, SSTF proved to be the more efficient algorithm for the selected input, reducing the total head movement by approximately 50.26%. However, despite its lower movement cost, SSTF has a potential drawback: requests located far from the current head position may experience long waiting times or starvation. Therefore, the choice of disk scheduling algorithm depends on whether minimizing head movement or ensuring fair request servicing is the primary concern.

GitHub:

https://github.com/AbirOPS2/CSE402-Operating-System-Lab/tree/main/Lab-07-Disk-Scheduling-SSTF-FCFS